

Top 20 JavaScript Questions for Middle-Level Developers (2026)

Curated interview questions for developers with 2–4 years of experience — deeper than junior fundamentals, focused on async internals, memory, patterns, modules, performance, and production trade-offs.

Table of Contents

1. [Explain microtasks vs macrotasks in the event loop](#)
2. [What are `call`, `apply`, and `bind`? When do you use each?](#)
3. [What is the difference between debounce and throttle?](#)
4. [What causes memory leaks in JavaScript and how do you prevent them?](#)
5. [When do you use `WeakMap` / `WeakSet` instead of `Map` / `Set`?](#)
6. [What is the difference between `prototype` and `\_\_proto\_\_`?](#)
7. [Compare `Promise.all`, `Promise.allSettled`, `Promise.race`, and `Promise.any`](#)
8. [How do you cancel async operations with `AbortController`?](#)
9. [What is the difference between currying and partial application?](#)

10. [How does memoization work and when should you use it?](#)
 11. [Explain the module pattern and data encapsulation in JavaScript](#)
 12. [What are the practical differences between CommonJS and ES modules?](#)
 13. [How do dynamic imports and code splitting work?](#)
 14. [What is tree shaking and what makes code tree-shakeable?](#)
 15. [What is prototype pollution and how do you mitigate it?](#)
 16. [How do generators and async generators work?](#)
 17. [What are `Proxy` and `Reflect` used for in real code?](#)
 18. [How do you diagnose and optimize JavaScript performance in the browser?](#)
 19. [What client-side security risks should mid-level developers know?](#)
 20. [How do you test asynchronous JavaScript effectively?](#)
-

1. Explain microtasks vs macrotasks in the event loop

Answer: The event loop processes work in phases. After each **macrotask** (also called a task), the engine drains the entire **microtask queue** before picking the next macrotask.

Macrotasks: `setTimeout`, `setInterval`, I/O callbacks, UI rendering, `message` events

Microtasks: Promise `.then` / `.catch` / `.finally`, `queueMicrotask`, `MutationObserver`, `await` continuations

```
console.log("1 sync");

setTimeout(() => console.log("2 macrotask"), 0);

Promise.resolve()
  .then(() => console.log("3 microtask"))
  .then(() => console.log("4 microtask"));

queueMicrotask(() => console.log("5 microtask"));

console.log("6 sync");

// Output: 1, 6, 3, 5, 4, 2
```

Execution model:

Run synchronous code (call stack)



Drain ALL microtasks



Run ONE macrotask



(repeat: microtasks → macrotask)

Why mid-level devs need this:

- Explains why `await` runs before `setTimeout(fn, 0)`
- Prevents bugs when mixing DOM updates, Promises, and timers
- Explains "microtask starvation" — long microtask chains block rendering and input

Interview follow-up: `async function` bodies run synchronously until the first `await`; everything after `await` is scheduled as a microtask.

2. What are `call`, `apply`, and `bind`? When do you use each?

Answer: All three explicitly set `this` for a function call. They are essential for borrowing methods, constructor chaining, and partial application.

Method	Invokes immediately?	Arguments passed as
<code>call</code>	Yes	Comma-separated
<code>apply</code>	Yes	Array

Method	Invokes immediately?	Arguments passed as
<code>bind</code>	No — returns new fn	Comma-separated

```
function greet(greeting, punctuation) {
  return `${greeting}, ${this.name}${punctuation}`;
}

const user = { name: "Alex" };

greet.call(user, "Hello", "!");           // "Hello, Alex!"
greet.apply(user, ["Hi", "."]);           // "Hi, Alex."

const sayHello = greet.bind(user, "Hello");
sayHello("!!!");                          // "Hello, Alex!!!"
```

Real-world use cases:

```
// Borrow array methods on array-like objects
const args = { 0: "a", 1: "b", length: 2 };
Array.prototype.slice.call(args); // ["a", "b"]

// Preserve this in event handlers (before arrow functions
were common)
class Counter {
  constructor() {
    this.count = 0;
    this.el = document.querySelector("#btn");
    this.el.addEventListener("click",
this.increment.bind(this));
  }
}
```

```
increment() { this.count++; }  
  
}  
  
// Partial application  
const multiply = (a, b) => a * b;  
const double = multiply.bind(null, 2);  
double(5); // 10
```

2026 note: Arrow functions and class fields reduce `bind` usage, but you still need these methods when working with legacy APIs, libraries, or explicit `this` control.

3. What is the difference between debounce and throttle?

Answer: Both limit how often a function runs — common for scroll, resize, search input, and API calls.

Technique	Behavior	Best for
Debounce	Runs once after activity stops for N ms	Search input, form validation
Throttle	Runs at most once every N ms during activity	Scroll, resize, mousemove

Debounce implementation:

```
function debounce(fn, delay) {
  let timerId;

  return function (...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => fn.apply(this, args),
delay);
  };
}

const search = debounce((query) => {
  fetch(`/api/search?q=${query}`);
}, 300);

input.addEventListener("input", (e) =>
search(e.target.value));
// API called only after user stops typing for 300ms
```

Throttle implementation:

```
function throttle(fn, limit) {
  let inThrottle = false;

  return function (...args) {
    if (inThrottle) return;
    fn.apply(this, args);
    inThrottle = true;
    setTimeout(() => { inThrottle = false; }, limit);
  };
}

const onScroll = throttle(() => {
```

```
updateStickyHeader();  
}, 100);  
window.addEventListener("scroll", onScroll);  
// Runs at most every 100ms while scrolling
```

Leading vs trailing debounce: Trailing (above) waits until pause. Leading fires immediately then waits — useful for button double-click prevention.

Interview tip: Be ready to implement either from scratch and explain trade-offs (debounce adds latency; throttle may skip the final state).

4. What causes memory leaks in JavaScript and how do you prevent them?

Answer: JavaScript has garbage collection, but objects stay in memory if they remain **reachable**. Leaks happen when references are held longer than intended.

Common causes:

Cause	Example
Detached DOM nodes	Removed elements still referenced in JS

Cause	Example
Forgotten event listeners	Listener keeps component + closure alive
Global variables	Accidental <code>window.cache = hugeData</code>
Closures holding large data	Callback captures entire scope
Timers / intervals	<code>setInterval</code> never cleared
Caches without bounds	Map grows forever

```
// LEAK: detached DOM + listener
function setup() {
  const btn = document.querySelector("#btn");
  const data = new Array(1_000_000).fill("x"); // large
  payload

  btn.addEventListener("click", () => {
    console.log(data.length); // closure keeps data + btn
    alive
  });

  document.body.removeChild(btn); // DOM gone, but JS
  still references it
}

// FIX: cleanup
```

```
function setup() {  
  const btn = document.querySelector("#btn");  
  const handler = () => console.log("clicked");  
  
  btn.addEventListener("click", handler);  
  
  return () => {  
    btn.removeEventListener("click", handler);  
  };  
}
```

Prevention checklist:

1. Remove event listeners in SPA teardown (`useEffect` cleanup, `onDestroy`)
2. Clear timers: `clearTimeout`, `clearInterval`
3. Use `WeakMap` for metadata tied to DOM nodes
4. Cap cache size (LRU pattern)
5. Avoid storing DOM references in module-level variables
6. Use Chrome DevTools **Memory** tab — heap snapshots, detached nodes report

Node.js extra: Unclosed DB connections, growing `EventEmitter` listener counts, unbounded `global` state.

5. When do you use `WeakMap` / `WeakSet` instead of `Map` / `Set`?

Answer: `WeakMap` and `WeakSet` hold **weak references** to objects — if nothing else references the key object, it can be garbage-collected. Keys must be objects (not primitives).

Feature	<code>Map</code> / <code>Set</code>	<code>WeakMap</code> / <code>WeakSet</code>
Keys	Any type	Objects only
Iterable	Yes	No
Size property	Yes	No
GC of unused keys	No — holds strong	Yes — weak references

```
// Private metadata on DOM nodes — no memory leak when
node is removed
const metadata = new WeakMap();

function attachWidget(el, config) {
  metadata.set(el, config);
}

function getConfig(el) {
  return metadata.get(el);
}

// When el is removed from DOM and unreferenced, metadata
entry is GC'd
```

```
// Caching computed results per object instance
const cache = new WeakMap();

function expensiveCalc(obj) {
  if (cache.has(obj)) return cache.get(obj);
  const result = /* heavy work */ obj.data.reduce((a, b)
=> a + b, 0);
  cache.set(obj, result);
  return result;
}
```

When NOT to use WeakMap:

- You need to iterate all entries
- You need to know the count of stored items
- Keys are strings/numbers — use `Map`

WeakSet use case: Track objects you've already processed without preventing GC.

```
const processed = new WeakSet();

function processOnce(obj) {
  if (processed.has(obj)) return;
  processed.add(obj);
  // ... do work
}
```

6. What is the difference between `prototype` and `__proto__`?

Answer: These are two sides of the same prototype chain mechanism.

- `prototype` — property on **constructor functions** (and classes). The object that will become the `[[Prototype]]` of instances created with `new`.
- `__proto__` — deprecated accessor to an object's `[[Prototype]]` (internal link). Prefer `Object.getPrototypeOf()` / `Object.setPrototypeOf()`.

```
function Person(name) {
  this.name = name;
}

Person.prototype.greet = function () {
  return `Hi, ${this.name}`;
};

const alex = new Person("Alex");

// Prototype chain:
// alex → Person.prototype → Object.prototype → null

alex.__proto__ === Person.prototype;           // true
Object.getPrototypeOf(alex) === Person.prototype; // true
Person.prototype.constructor === Person;       // true
```

How `new` works (simplified):

1. Create empty object
2. Set its `[[Prototype]]` to `Constructor.prototype`
3. Run constructor with `this` = new object
4. Return object (unless constructor returns an object)

```
function createInstance(Constructor, ...args) {
  const obj = Object.create(Constructor.prototype);
  const result = Constructor.apply(obj, args);
  return result instanceof Object ? result : obj;
}
```

Interview trap: `typeof` on a class instance is `"object"`, methods live on the prototype (shared), own properties live on the instance. `hasOwnProperty` vs `in` operator tests own vs inherited properties.

7. Compare `Promise.all`, `Promise.allSettled`, `Promise.race`, and `Promise.any`

Answer: These compose multiple Promises with different failure semantics.

Method	Resolves when	Rejects when	Result shape
<code>Promise.all</code>	All fulfill	First rejection	Array of values
<code>Promise.allSettled</code>	All settle (any state)	Never	Array of <code>{status, value reason}</code>
<code>Promise.race</code>	First to settle	First rejection (if first)	Single value
<code>Promise.any</code>	First fulfillment	All reject (AggregateError)	Single value

```

const ok = Promise.resolve("A");
const fail = Promise.reject("B");
const slow = new Promise(r => setTimeout(() => r("C"), 100));

await Promise.all([ok, slow]);           // ["A", "C"]
await Promise.all([ok, fail]);           // rejects "B" - fail-fast

await Promise.allSettled([ok, fail]);
// [{ status: "fulfilled", value: "A" },
//   { status: "rejected", reason: "B" }]

```

```
await Promise.race([fail, slow]);    // rejects "B" –  
first to settle  
await Promise.any([fail, slow]);    // "C" – first to  
fulfill
```

When to use what:

- **Promise.all** — parallel fetches where one failure should abort everything (load dashboard widgets)
- **Promise.allSettled** — batch operations where you need every result (bulk delete with per-item status)
- **Promise.race** — timeouts, first-responder wins
- **Promise.any** — fallback sources (try CDN A, CDN B, CDN C — first success wins)

```
// Timeout pattern with Promise.race  
function fetchWithTimeout(url, ms = 5000) {  
  const timeout = new Promise((_, reject) =>  
    setTimeout(() => reject(new Error("Timeout")), ms)  
  );  
  return Promise.race([fetch(url), timeout]);  
}
```

8. How do you cancel async operations with

AbortController ?

Answer: `AbortController` provides a standard signal to cancel `fetch`, streams, event listeners, and custom async logic.

```
const controller = new AbortController();
const { signal } = controller;

async function loadUsers() {
  try {
    const res = await fetch("/api/users", { signal });
    return await res.json();
  } catch (err) {
    if (err.name === "AbortError") {
      console.log("Request cancelled");
      return null;
    }
    throw err;
  }
}

const promise = loadUsers();

// Cancel from UI — e.g. component unmount or "Cancel"
button
controller.abort();
```

Combining multiple requests:

```
const controller = new AbortController();

const [users, posts] = await Promise.all([
  fetch("/api/users", { signal: controller.signal
}).then(r => r.json()),
```

```

    fetch("/api/posts", { signal: controller.signal
  }).then(r => r.json()),
]);

controller.abort(); // cancels both

```

Custom async work:

```

function poll(url, intervalMs, signal) {
  return new Promise((resolve, reject) => {
    const id = setInterval(async () => {
      if (signal.aborted) {
        clearInterval(id);
        reject(signal.reason ?? new
DOMException("Aborted", "AbortError"));
        return;
      }
      const data = await fetch(url, { signal }).then(r =>
r.json());
      if (data.ready) {
        clearInterval(id);
        resolve(data);
      }
    }, intervalMs);

    signal.addEventListener("abort", () =>
clearInterval(id), { once: true });
  });
}

```

React pattern: Create controller in `useEffect`, abort in cleanup.

2026: Supported everywhere — prefer over deprecated axios cancel tokens or manual flags.

9. What is the difference between currying and partial application?

Answer: Both transform functions to pre-fill arguments, but they differ in arity and flexibility.

Currying — transforms `f(a, b, c)` into `f(a)(b)(c)`. Each call takes **one** argument until all are provided.

Partial application — fixes **some** arguments upfront, returns a function accepting the rest.

```
// Currying
const add = (a) => (b) => (c) => a + b + c;
add(1)(2)(3); // 6

const curriedMultiply = (a) => (b) => a * b;
const double = curriedMultiply(2);
double(5); // 10

// Partial application
const multiply = (a, b, c) => a * b * c;
const doublePartial = (b, c) => multiply(2, b, c);
doublePartial(3, 4); // 24
```

```
// Or with bind
const doubleBind = multiply.bind(null, 2);
doubleBind(3, 4); // 24
```

Generic curry helper:

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return (...next) => curried(...args, ...next);
  };
}

const sum = curry((a, b, c) => a + b + c);
sum(1)(2)(3);    // 6
sum(1, 2)(3);    // 6
sum(1)(2, 3);    // 6
```

When it matters: Currying enables function composition pipelines; partial application is simpler when you just need a specialized function from a general one.

10. How does memoization work and when should you use it?

Answer: Memoization caches function results keyed by arguments to avoid redundant computation.

```

function memoize(fn, { maxSize = 100 } = {}) {
  const cache = new Map();

  return function (...args) {
    const key = JSON.stringify(args); // use structured
    key for production

    if (cache.has(key)) return cache.get(key);

    const result = fn.apply(this, args);

    if (cache.size >= maxSize) {
      const firstKey = cache.keys().next().value;
      cache.delete(firstKey); // simple eviction
    }

    cache.set(key, result);
    return result;
  };
}

const fib = memoize(function fib(n) {
  if (n <= 1) return n;
  return fib(n - 1) + fib(n - 2);
});

fib(50); // fast - without memoization this is exponential

```

When to use:

- Pure functions with expensive computation (sorting, parsing, recursive algorithms)
- Repeated calls with same inputs (React `useMemo` is React's version of this)

When NOT to use:

- Functions with side effects
- Arguments that are objects/arrays (reference identity breaks naive caching)
- Memory-constrained environments without cache bounds
- Cheap functions — memoization overhead exceeds benefit

Production considerations:

- Use a proper cache key (`hash` , `Map` with serialized args, or `WeakMap` keyed by object)
- Set max cache size or TTL
- Clear cache on data invalidation

11. Explain the module pattern and data encapsulation in JavaScript

Answer: Before ES modules, the **module pattern** used IIFEs and closures to create private state with a public API.

```

const UserModule = (function () {
  // Private – not accessible outside
  let users = [];
  let nextId = 1;

  function validate(name) {
    return typeof name === "string" && name.trim().length
    > 0;
  }

  // Public API
  return {
    add(name) {
      if (!validate(name)) throw new Error("Invalid
name");
      const user = { id: nextId++, name: name.trim() };
      users.push(user);
      return user;
    },

    getAll() {
      return [...users]; // return copy – protect internal
state
    },

    findById(id) {
      return users.find(u => u.id === id) ?? null;
    },
  };
})();

```

```
UserModule.add("Alex");  
UserModule.users; // undefined – private
```

Revealing module pattern — explicitly map private functions to public object:

```
const Calculator = (() => {  
  const add = (a, b) => a + b;  
  const multiply = (a, b) => a * b;  
  
  return { add, multiply }; // reveal selected methods  
})();
```

Modern equivalent with ES modules:

```
// userStore.js  
let users = [];  
let nextId = 1;  
  
export function addUser(name) { /* ... */ }  
export function getAllUsers() { return [...users]; }  
// users is module-scoped – not exported, effectively  
private
```

2026 perspective: Use ES modules in new code. Understand IIFE/module pattern for reading legacy codebases and interview questions about encapsulation without classes.

12. What are the practical differences between CommonJS and ES modules?

Answer: Two module systems coexist in JavaScript — especially in Node.js.

Aspect	CommonJS (<code>require</code>)	ES Modules (<code>import</code>)
Loading	Dynamic, synchronous	Static analysis; async in Node
Syntax	<code>require()</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Top-level await	No	Yes (ESM only)
Tree shaking	Limited	Yes
<code>this</code> at top level	<code>module.exports</code>	<code>undefined</code> (strict)
Circular deps	Copy of partial exports	Live bindings

```
// CommonJS
const fs = require("fs");
module.exports = { read: fs.readFileSync };
module.exports.helper = () => {};
```

```
// Dynamic require at runtime
const name = "lodash";
const lib = require(name);

// ESM
import fs from "node:fs";
export { readFileSync } from "node:fs";
export default function helper() {}

// Dynamic import — returns Promise
const mod = await import("./heavy-module.js");
```

Interop in Node.js:

- `"type": "module"` in `package.json` → `.js` files are ESM
- Import CJS from ESM: `import pkg from "cjs-pkg"` (default export = `module.exports`)
- Require ESM from CJS: not directly — use dynamic `import()`

Interview angle: ESM enables tree shaking and static analysis; CJS remains in older Node libraries. Bundlers (Vite, webpack) mostly emit ESM for browsers.

13. How do dynamic imports and code splitting work?

Answer: Dynamic `import()` loads a module at runtime, returning a Promise. Bundlers use it to create separate **chunks** loaded on demand.

```
// Lazy-load a heavy chart library only when needed
async function showChart(data) {
  const { Chart } = await import("chart.js");
  return new Chart(document.getElementById("canvas"), { /*
config */ });
}

// Route-based splitting (conceptual – frameworks wrap
this)
const routes = {
  "/dashboard": () => import("./pages/Dashboard.js"),
  "/settings": () => import("./pages/Settings.js"),
};

async function navigate(path) {
  const loadPage = routes[path];
  if (!loadPage) return;

  const module = await loadPage();
  module.render(document.getElementById("app"));
}
```

What the bundler produces:

```
main.js           ← initial bundle (small, fast first
paint)
dashboard.chunk.js
```

```
settings.chunk.js  
chart-vendor.chunk.js
```

Benefits:

- Smaller initial load → faster TTI (Time to Interactive)
- Load features only when user needs them
- Parallel chunk downloads in HTTP/2+

Patterns:

- **Route-level splitting** — each page is a chunk
- **Component-level splitting** — modals, editors, maps
- **Preload / prefetch** — `<link rel="modulepreload">` or webpack magic comments

```
// Vite / webpack hint: prefetch likely next route  
import(/* webpackPrefetch: true */ "../pages/Checkout.js");
```

14. What is tree shaking and what makes code tree-shakeable?

Answer: **Tree shaking** is dead-code elimination at build time — removing unused exports so they never reach the browser bundle.

Requirements for tree shaking:

1. **ES modules** with static `import` / `export`
2. **Side-effect-free** modules (or marked in `package.json`)
3. Bundler support (Rollup, esbuild, webpack, Vite)

```
// utils.js
export function used() { return "keep me"; }
export function unused() { return "remove me"; }

// app.js
import { used } from "./utils.js";
console.log(used());
// unused() is never imported → dropped from production
bundle
```

What breaks tree shaking:

```
// Side effect on import – entire module kept
import "./polyfills.js"; // runs top-level code

// CommonJS – bundler can't statically analyze
const utils = require("./utils");

// Barrel file re-exporting everything
// index.js: export * from "./a"; export * from "./b"; ...
// import { oneFn } from "./index" – may pull in all re-
exports
import { oneFn } from "./a.js"; // prefer direct imports
```

`package.json` **sideEffects:**

```
{
  "sideEffects": false
}
// or list files with side effects:
// "sideEffects": ["*.css", "./src/polyfills.js"]
```

Interview follow-up: Tree shaking happens at build time; minification (Terser/esbuild) removes dead code within kept functions. Both reduce bundle size.

15. What is prototype pollution and how do you mitigate it?

Answer: Prototype pollution is a vulnerability where an attacker modifies `Object.prototype` (or other builtins), affecting **all** objects in the runtime.

```
// Vulnerable merge — DO NOT write this
function merge(target, source) {
  for (const key in source) {
    target[key] = source[key]; // unsafe if key is
    "__proto__"
  }
  return target;
}

const payload = JSON.parse('{ "__proto__": {"isAdmin":
true}}');
const config = {};
```

```
merge(config, payload);

const user = {};
console.log(user.isAdmin); // true - polluted! Every
object inherits isAdmin
```

Attack vectors:

- Unsafe deep merge / clone utilities
- Parsing untrusted JSON into objects without validation
- `Object.assign` with crafted input in some patterns

Mitigation:

```
// 1. Use null-prototype objects for maps
const safe = Object.create(null);
safe["__proto__"] = "ignored as normal key"; // no
pollution

// 2. Validate keys
const FORBIDDEN = new Set(["__proto__", "constructor",
"prototype"]);
function safeSet(obj, key, value) {
  if (FORBIDDEN.has(key)) throw new Error("Invalid key");
  obj[key] = value;
}

// 3. Use Map instead of plain objects for untrusted keys
const store = new Map();
store.set(userKey, value);
```

```
// 4. Freeze prototypes in security-sensitive apps
(defense in depth)
Object.freeze(Object.prototype);

// 5. Use established libraries (lodash merge patched in
4.17.12+)
// Keep dependencies updated
```

2026 awareness: Prototype pollution is a common npm supply-chain and CVE topic. Code review any utility that recursively assigns properties from user input.

16. How do generators and async generators work?

Answer: Generators (`function*`) are pausable functions that yield multiple values. **Async generators** (`async function*`) yield Promises — useful for streaming data.

Sync generator:

```
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;
  }
}

const gen = idGenerator();
gen.next(); // { value: 1, done: false }
```



```
gen.next(); // { value: 2, done: false }

function* range(start, end) {
  for (let i = start; i <= end; i++) yield i;
}

for (const n of range(1, 3)) console.log(n); // 1, 2, 3
```

Async generator — paginated API stream:

```
async function* fetchPages(baseUrl) {
  let page = 1;
  let hasMore = true;

  while (hasMore) {
    const res = await fetch(`${baseUrl}?page=${page}`);
    const data = await res.json();
    yield data.items;
    hasMore = data.hasMore;
    page++;
  }
}

for await (const items of fetchPages("/api/products")) {
  renderProducts(items); // process page by page — low
  memory
}
```

Use cases:

Feature	Use case
Sync generators	Lazy sequences, custom iterables, coroutine-style state machines
Async generators	Streaming HTTP, paginated APIs, reading file chunks
<code>yield*</code>	Delegate to another generator

vs `async/await`: Generators excel when you produce a **sequence** over time. `async/await` excels when you have a fixed chain of async steps.

17. What are `Proxy` and `Reflect` used for in real code?

Answer: `Proxy` wraps an object and intercepts operations (get, set, delete, etc.). `Reflect` provides default behavior for those operations — used inside Proxy traps.

```
function createValidatedUser(target) {
  const validator = {
    set(obj, prop, value) {
      if (prop === "age" && (typeof value !== "number" ||
value < 0)) {
        throw new TypeError("age must be a non-negative
number");
      }
    }
  }
}
```

```

    if (prop === "email" && !value.includes("@")) {
      throw new TypeError("invalid email");
    }
    return Reflect.set(obj, prop, value);
  },
};

return new Proxy(target, validator);
}

const user = createValidatedUser({ name: "Alex", age: 30
});
user.age = -1; // TypeError

```

Real-world uses:

- **Reactive frameworks** — Vue 3 reactivity is built on Proxy
- **Validation layers** — reject invalid property assignments
- **Logging / debugging** — trace all property access
- **Default values** — return fallback for missing keys
- **Immutable wrappers** — throw on mutation attempts

```

// Default values trap
const withDefaults = (target, defaults) =>
  new Proxy(target, {
    get(obj, prop) {
      return prop in obj ? obj[prop] : defaults[prop];
    },
  });

```

```
const config = withDefaults({}, { theme: "light", lang: "en" });  
config.theme; // "light"
```

Limitations: Proxies do not intercept all internal slots (e.g. private fields `#x`), and some built-ins behave specially. Not a drop-in for all objects.

18. How do you diagnose and optimize JavaScript performance in the browser?

Answer: Mid-level developers should move beyond "make it faster" and use systematic profiling.

Diagnosis tools:

Tool	Purpose
Chrome Performance panel	Flame charts, long tasks, layout thrashing
Lighthouse	Core Web Vitals, actionable audits
Memory panel	Heap snapshots, detached DOM nodes
Network panel	Bundle size, waterfall, caching

Tool	Purpose
<code>performance.mark/measure</code>	Custom timing in code

Common bottlenecks and fixes:

```
// 1. Layout thrashing – interleaving reads and writes
// BAD
elements.forEach(el => {
  el.style.width = el.offsetWidth + 10 + "px"; // read +
write each iteration
});

// GOOD – batch reads, then batch writes
const widths = elements.map(el => el.offsetWidth);
elements.forEach((el, i) => {
  el.style.width = widths[i] + 10 + "px";
});

// 2. Expensive listeners – debounce/throttle (see Q3)

// 3. Large lists – virtualize (render only visible rows)

// 4. Main thread blocking – Web Workers for heavy
computation
const worker = new Worker("hash-worker.js");
worker.postMessage(largeDataset);
worker.onmessage = (e) => updateUI(e.data);
```

Rendering pipeline awareness:

JavaScript → Style → Layout → Paint → Composite

- Changing `transform` / `opacity` → often compositor-only (cheap)
- Changing `width`, `top`, `margin` → triggers layout (expensive)
- Prefer `requestAnimationFrame` for visual updates tied to display refresh

Bundle optimization: Code splitting, tree shaking, lazy loading, analyze with `rollup-plugin-visualizer` or Vite's bundle analyzer.

Core Web Vitals (2026): LCP (loading), INP (interactivity — replaced FID), CLS (visual stability).

19. What client-side security risks should mid-level developers know?

Answer: Frontend code is fully visible and manipulable. Security is **defense in depth** — never trust the client alone.

XSS (Cross-Site Scripting) — injecting malicious scripts via user content:

```
// VULNERABLE
element.innerHTML = userComment;

// SAFE — text only
element.textContent = userComment;
```

```
// If HTML is required – sanitize
import DOMPurify from "dompurify";
element.innerHTML = DOMPurify.sanitize(userComment);
```

Never store secrets in frontend code:

```
// WRONG – visible in bundle and DevTools
const API_KEY = "sk-live-abc123";
fetch(`/api?key=${API_KEY}`);

// RIGHT – backend proxy holds the key
fetch("/api/internal-service"); // server adds credentials
```

Other risks:

Risk	Mitigation
XSS via <code>innerHTML</code> , <code>eval</code> , <code>document.write</code>	Sanitize, CSP, avoid <code>eval</code>
CSRF	SameSite cookies, CSRF tokens (server-side)
Prototype pollution	Safe merge, <code>Object.create(null)</code>
Sensitive data in localStorage	Use httpOnly cookies for tokens

Risk	Mitigation
Open redirects	Validate redirect URLs server-side
Dependency vulnerabilities	<code>npm audit</code> , Dependabot, lockfiles

Content Security Policy (CSP):

```
<meta http-equiv="Content-Security-Policy"
      content="default-src 'self'; script-src 'self';
      object-src 'none'">
```

Restricts script sources — mitigates XSS even if injection occurs.

Key principle: Validate and authorize on the **server**. Client validation is UX, not security.

20. How do you test asynchronous JavaScript effectively?

Answer: Async tests must **await** results, assert on both success and failure paths, and isolate side effects.

Promises with Vitest / Jest:

```
import { describe, it, expect, vi } from "vitest";
import { fetchUser } from "../api.js";
```



```

describe("fetchUser", () => {
  it("returns user data on success", async () => {
    global.fetch = vi.fn().mockResolvedValue({
      ok: true,
      json: () => Promise.resolve({ id: 1, name: "Alex"
    })),
  });

  const user = await fetchUser(1);
  expect(user.name).toBe("Alex");
  expect(fetch).toHaveBeenCalledWith("/api/users/1",
expect.any(Object));
  });

  it("throws on HTTP error", async () => {
    global.fetch = vi.fn().mockResolvedValue({ ok: false,
status: 404 });

    await expect(fetchUser(1)).rejects.toThrow("404");
  });
});

```

Fake timers for debounce/throttle:

```

import { vi } from "vitest";

it("debounces search", () => {
  vi.useFakeTimers();
  const fn = vi.fn();
  const debounced = debounce(fn, 300);

```

```

debounced();
debounced();
debounced();
expect(fn).not.toHaveBeenCalled();

vi.advanceTimersByTime(300);
expect(fn).toHaveBeenCalledTimes(1);

vi.useRealTimers();
});

```

Testing patterns:

Pattern	When
<code>async/await</code> in tests	Most async code
<code>waitFor</code> (Testing Library)	DOM updates after async
Mock <code>fetch</code> / HTTP layer	API unit tests
MSW (Mock Service Worker)	Integration tests without real backend
Spy on callbacks	Event-driven code
<code>AbortController</code> in tests	Verify cancellation on unmount

```

// MSW – intercept network at service worker level
import { http, HttpResponse } from "msw";

```

```
import { setupServer } from "msw/node";

const server = setupServer(
  http.get("/api/users/:id", ({ params }) =>
    HttpResponse.json({ id: params.id, name: "Alex" })
  )
);

beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());
```

Anti-patterns:

- Fixed `setTimeout` in tests instead of fake timers
- Not awaiting async assertions (`expect(promise).resolves...` without `await`)
- Testing implementation details instead of behavior
- Shared mutable state between tests

Quick Study Tips

1. **Implement from scratch** — debounce, throttle, `Promise.all`, memoize, curry.
2. **Draw the event loop** — microtasks vs macrotasks on a whiteboard.

3. **Debug a memory leak** — create one intentionally, fix it with DevTools.
4. **Read bundle output** — understand what Vite/webpack actually ships.
5. **Practice system design lite** — "How would you build autocomplete?" ties together debounce, fetch, AbortController, caching, and testing.